Prvi popravni test iz
Algoritama i struktura podataka

Prvi praktični test održan 19.01.2024.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 30 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na nazbačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2023_PopravniTest1_GrupaA_IT1g2021**
8. **Minimalni broj bodova koji se mogu osvojiti iz svakog pojedinačnog zadatka je 5**.
9. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Brisanje elemenata iz reda (Queue Dequeue)*

Napisati metodu koja implementira standardan način za brisanje elemenata iz reda.

Dodatne napomene za rešavanje zadatka:

- Algoritam napisati u okviru zakomentarisane metode ***dequeue?(int value).***
- Pomenuta metoda nalazi se u okviru klase ***Queue*** koja se nalazi u pridruženom template-u.

Obezbediti kod u Main metodi za testiranje algoritma.

Zadatak 2 – *Neparan broj parnih vrednosti u redovima matrice*

Napisati algoritam koji kao ulaz ima matricu brojeva NxM, broj redova i broj kolona te matrice, i koji proverava da li se u redu elementi nalazi neparan broj parnih vrednosti. Algoritam vraća niz sa onoliko elemenata koliko je redova u marici, tako da je na i-tom indeksu ovog niza jedinica ukoliko je u i-tom redu matrice neparan broj parnih vrednosti, u suprotnom na i-tom indeksu izlaznog niza treba biti nula.

a) Implementirati algoritam.
b) U *Main* metodi obezbediti testiranje, pozivom metode sa parametrima koji su dati u primerima.
c) 0 se posmatra odvojeno – nije ni paran, ni neparan broj.

Dodatni zahtevi za rešavanje zadatka:

- Zahtevan potpis metode: ***public static int[] checkOddEvenRows(int[,] matrix, int rows, int cols)***

Primer:

- Input: $\begin{bmatrix} 2 & 0 & 4 & 0 \\ 3 & 5 & 7 & 9 \\ 0 & 1 & 1 & 2 \\ 1 & 2 & 3 & 4 \end{bmatrix}$, 4, 4
- Output: [0, 0, 1, 0]

- Input: $\begin{bmatrix} 2 & 4 & 4 \\ 1 & 4 & 3 \\ 1 & 1 & 2 \end{bmatrix}$, 3, 3
- Output: [1, 1, 1]

Zadatak 3 – *Pronalaženje najduže sekvence rastućih vrednosti*

Napisati algoritam koji vraća najdužu sekvencu (niz) te jednostruko spregnute liste u kojoj su vrednosti u rastućem poretku. Ukoliko ne postoji sekvenca sa rastućim poretkom vratiti **null**.

Dodatne napomene za rešavanje zadatka:

a) Zahtevan potpis metode: ***public static int[]? findLargestAscendingSequence(LinkedList list)***

Primer:

- Input: ***LinkedList: 1 -> 2 -> 6 -> 0 -> 3***
- Output: ***[1, 2, 6]***

- Input: ***LinkedList: 2 -> 0 -> 6 -> 8 -> 4 -> 2 -> 3 -> 1***
- Output: ***[0, 6, 8]***

- Input: ***LinkedList: 6 -> 5 -> 4 -> 3***
- Output: ***null***